



Jose Parreño Garcia <joparga3@gmail.com>

[TEST wur - Paid] BETTER TITLE

1 mensaje

Jose Parreño Garcia from Senior Data Science Lead <joseparreogarcia@substack.com>
Para: joparga3@gmail.com

31 de marzo de 2026 a las 20:38

[View in browser](#)

The Senior Data Science Lead newsletter

Hi! It's Jose from Senior Data Science Lead newsletter, where I write about senior data science leadership, sharing hard-earned lessons from building teams, systems, and real-world machine learning.

Because you are a paid subscriber, you are directly supporting this work — and you get full access to the practical tools built alongside the writing:

- *The full Data Storytelling course, focused on turning analysis into insights that land with executives. Because today, anyone can build a chart with AI, but very few people can build the right chart.*



Master Data Storytelling

JOSE PARREÑO GARCIA · NOVEMBER 27, 2025

[Read full story](#)

- *50+ Notion templates for hiring, onboarding, mentoring, and planning built for Data and Tech Leads. As a paid subscriber, you get **50% off** with code **PAIDSUBS-03PL**.*



50+ Data Science Templates

JOSE PARREÑO GARCIA · APRIL 8, 2025

[Read full story](#)

And as always, thank you for supporting the newsletter — your support is what keeps it independent, practical, and focused on real-world impact.

BETTER TITLE

Agentic coding is making speed cheap, but this is a nightmare for team leads.

JOSE PARREÑO GARCIA
MAR 31 · PAID · DRAFT



READ IN APP ↗

One of my direct reports came back from the weekend buzzing. Over two days, working with Claude as a coding partner, he had built and iterated on a new deep learning model, testing features we had been wanting to explore for months but never had the bandwidth to properly prototype. The results were genuinely exciting, so of course, we immediately started talking about getting it into a live A/B test.

But, that A/B test took more than three weeks to reach production.

Don't get me wrong — 3 weeks is actually fast. In the pre-agentic world, what he built over the weekend would have taken most of a month just to reach the same state. The speed was real. But that is not how the conversation felt. The prototype had taken two days. The integration work — pipelines, edge cases, testing, infrastructure, ownership — looked like a wall next to it.

This is the tension at the heart of agentic coding for anyone managing a technical team. The old iron triangle of speed, quality, and scale was a useful balancer precisely because all 3 sides were in tension. That balance has been disrupted. Agentic tools have collapsed one side — the early phase, the exploration, the “let's try this and see” — without touching the others.

If you are managing teams and feeling something similar, I am sure you will identify with my own experiences living through this transition.

✓ Subscribed

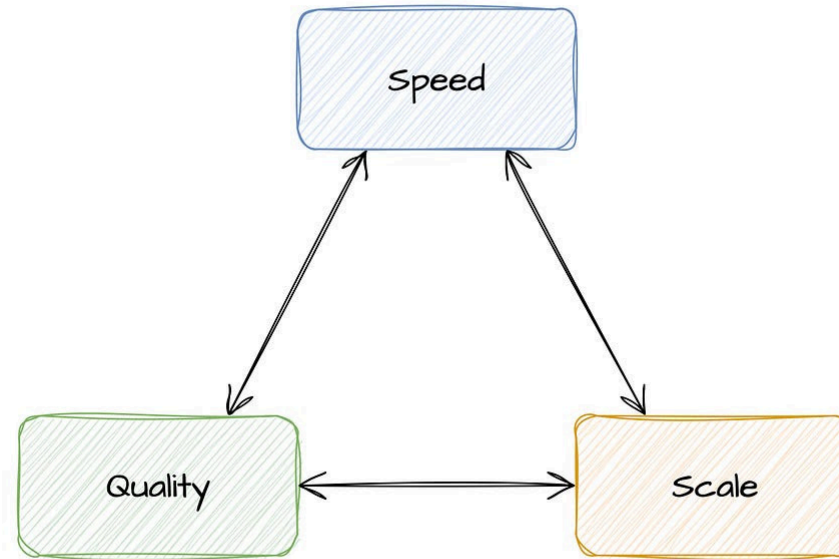
What will this blog cover?

- **The iron triangle of tech management.** For most of tech's professional history, speed, quality, and scope had to compete against each other. DevOps compressed the cycle a bit, but the underlying logic stay the same: you couldn't all 3 at the same time.
- **When speed stopped being the constraint.** Agentic coding has made the early phase of development — getting to “impressive enough to show” — radically cheaper. The phase that follows has not changed at all (or has gotten even worse).
- **5 ways agentic coding is disrupting tech leads.** Planning, the exploration-exploitation balance, managing up and validation. These dimensions are now in a completely different paradigm.
- **The contradictions your team is already living.** We build faster but, trust less. We explore more, but deliver less. We generate more code, but understand less of the system. These are not bugs, they are the natural shape of the reframed triangle.
- **What to do in the next 6 months.** 6 concrete practices that reduce the damage without pretending the tension resolves.

Let's get started!

The iron triangle of tech management (the triangle we thought we understood)

For most of tech's professional history, there was a stable iron triangle that tech leads could rely on.



You could optimise for speed, quality, or scale... but not all three simultaneously.

In practice, this translated into a lifecycle most teams implicitly understood. A proof of concept lived under different rules than a production system. For example: speed and learning mattered in the POC; but quality and reliability mattered in production. And maybe, the MVP existed somewhere in between.

The underlying assumption was really clear: **speed was earned**. Wasn't that why the 10x engineers existed? In came the Principal Data Scientist and she produced a working solution in 2-3 weeks that the team hadn't been able to solve in 3 months.

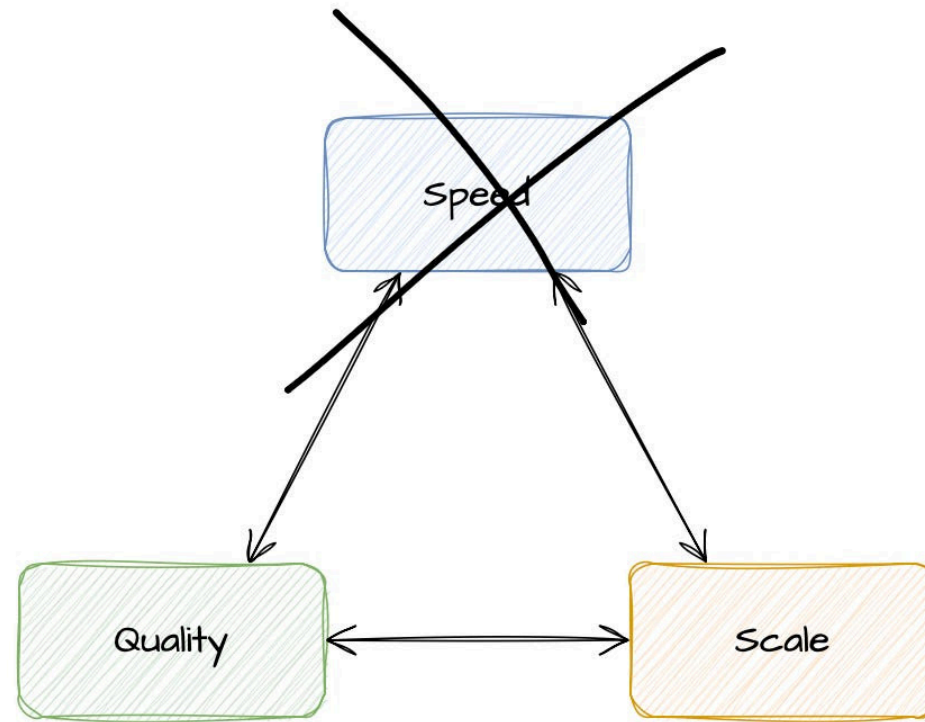
Building something **required time**, and that time was a natural forcing function. It forced prioritisation. It made trade-off conversations realistic. It gave teams a built-in reason to push back on things that were not worth the investment.

That forcing function has weakened, or in many cases, disappeared. And all because of how fast code can be produced.

When speed stopped being the constraint.

Agentic coding tools have made the early phase of software development extraordinarily fast. A prototype that once required two weeks of focused engineering can now be produced in an afternoon. A working proof of concept, complete with a connected model and a basic interface, can be on a shared screen by end of day. And, all of that, sometimes not even needing an engineer at all.

So yes, poof, speed is thrown out of the window.



This has changed the context in which tech leads operate. As [Gartner has noted](#), generative AI is explicitly reshaping the responsibilities of software engineering leaders — including how they manage team capacity, set expectations, and handle oversight of AI-generated output. I also really like how the [1,800 insight's describes it](#): *“the world of software development isn’t just getting a new set of tools; it’s experiencing a **tectonic plate shift** in its very foundations.”*

The middle management layer is being hit really hard. I speak from experience. Senior stakeholders are building things themselves now (or least,

a working PoC). They have used the tools. Some have built prototypes over weekends. When a VP has spent a Sunday watching an agent produce a working dashboard, their mental model of “how long should this take?” has been recalibrated — and the conversation about why something takes three months becomes much harder. Much much much harder.

The big issue is that there is now an asymmetry between 2 phases of work that used to move at similar speeds and now do not.

- **The cost of showing something has collapsed.** Even on deep analysis, Claude can produce insights that are really amazing. So yes, Claude can tell us: *“Hey, this, this and this other things are really wrong, and here are some complex papers backing the required solution that you guys need”*... but making these changes is not as easy.
- **The cost of making it real has not collapsed.** I’ve even had some senior execs telling me to port their code to production (you can imagine my face...). Right now, building production grade software code, machine learning systems that are not biased and analytics that embed business knowledge *are no where near the speed* that prototypes are being spun up.

In the past, both sides of the ratio (PoC vs Shipping) were measured in weeks, so the delta was manageable. When one side collapses to hours, the gap becomes a persistent source of friction — and the tech lead stands squarely in the middle of it.

5 ways agentic coding is disrupting tech leads

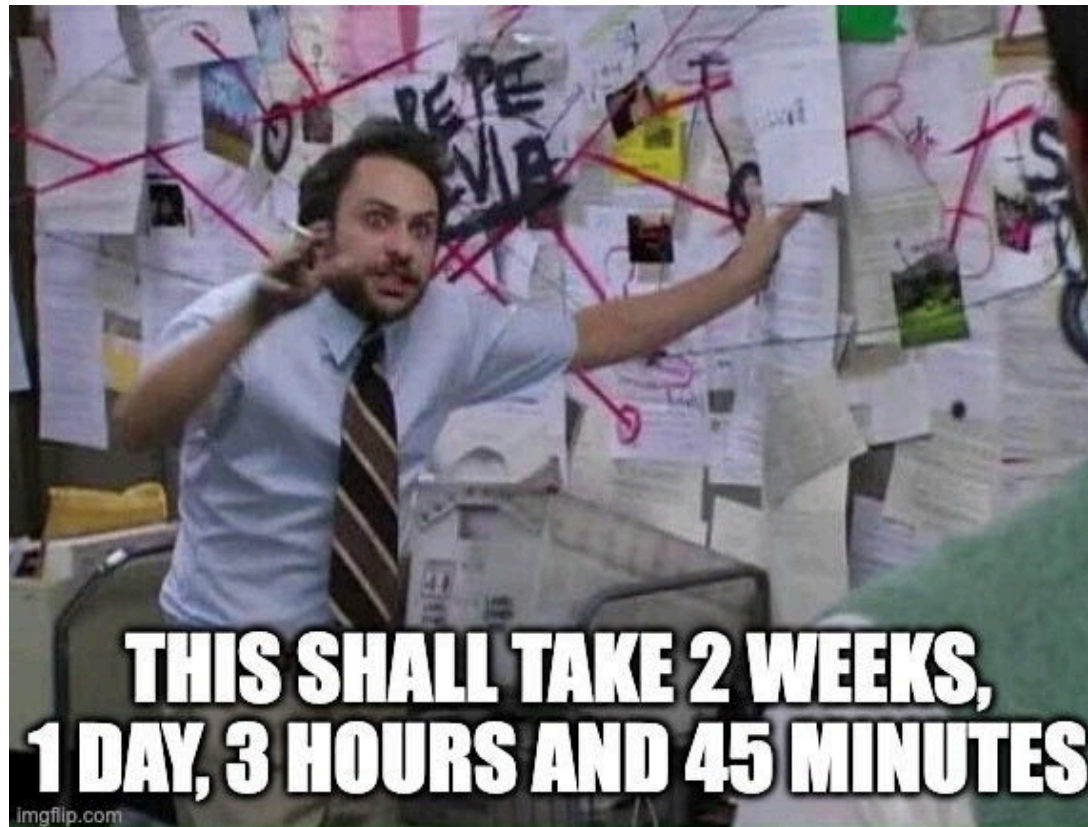
1. Planning becomes unreliable

The old model of effort estimation assumed a relatively stable relationship between task complexity and time. Senior engineers developed intuition for how long things take. In fact, senior engineers had lived through a way of working they were intimately knowledgeable on.

Now, agentic coding breaks the relationship. Everyone can see the speed at which things can be produced. But no one can now answer that task A will take 3 days or 3 weeks, because the agentic first pass could be fairly well estimated at 3 days, but putting that in production is totally unknown because agentic coding tools produce some much more code and artifacts to review, that its really really difficult to estimate how much something will take.

The 2025 Engineering Leadership Report, analysed by [GetQuotient](#), found that 60% of engineering leaders say AI has not significantly boosted team productivity. Only 6% reported gains above 30%. The productivity uplift is real but concentrated in specific task types — code generation, documentation — while integration and review work remains largely unchanged.

For planning, this creates overcommitment risk. Teams see fast early progress and extrapolate. The sprint ends with the feature “built” but not shippable. The planning model needs to shift: away from time-based estimation toward uncertainty management, with shorter feedback loops and explicit buffers for validation work that AI cannot compress.



2. Exploration vs exploitation flips

In the old world, exploration was expensive. Running a spike, prototyping an idea, trying a different architecture — these cost time that teams did not have in abundance. The default was exploitation: working on known problems with known tools toward committed outcomes. Exploration was tightly governed because it was genuinely costly.

Agentic coding has inverted this. Exploration is now cheap. A new approach can be tried in hours. An alternative library, a different model, a faster inference path — all of these can be prototyped before the end of the day.

What is not cheap is exploitation. Turning a prototype into a production system, integrating it with existing infrastructure, writing the tests, handling the failure modes — none of that has gotten faster. Engineers are now navigating code they did not write line by line, which adds its own layer of cognitive cost.

The result is fragmentation. Teams start **more** things but **fewer** things get finished. I can also see a extra context switching increases, and believe me, that is one of the killers for team productivity.

Which gets me to the idea that being DISCIPLINED is a new skill that tech leads need to adopt. The tech lead's job is not to encourage more exploration. It is to limit it explicitly.



3. Managing up becomes harder (and noisier)

This is the part of the role that few tech leads are formally trained for, and agentic coding has made it significantly harder.

As mentioned before, senior stakeholders have more direct exposure to AI capabilities than ever before and some have built things themselves. They are no longer asking whether AI can do something, because they have watched it do things. They assume AI can do anything (even *“simply refactoring the initial clunky code into production”*).

So, the conversation about why something takes three months becomes a credibility challenge. The tech lead is no longer explaining technical complexity to someone who has never touched the tools. They are explaining it to someone who has seen the tools work, and who reasonably wonders why the team cannot replicate what they saw on a Saturday afternoon.

The framing that tends to work is not “the demo was misleading.” It is: “the demo showed us what the ceiling looks like — my job is to make it reliable and something we can actually own.” Reliability and ownership are the words that resonate with leaders who have built things and watched them break.



4. The bottleneck shifts to validation and review

Ryan Salva, Senior Director of Product at Google, put it plainly in [a Stack Overflow interview](#): “Writing code was never really the bottleneck for shipping software. The bottlenecks were all of the other things surrounding it.”

Agentic coding has accelerated the thing that was not the bottleneck. Review, testing, integration, operational readiness — the actual bottlenecks — remain. And they are now under more pressure, because they receive more volume with the same resources.

The symptoms are familiar to anyone running a team right now: review fatigue, where engineers approve PRs they have not fully read; superficial testing, where coverage metrics look healthy but the scenarios that matter are not covered; slower feedback loops, where the review queue becomes a backlog.

Everyone is speaking that AI should review AI. I am not against that, but I have experienced how AI tricks tests and is not a reliable for quality and control. If the human has to define all these guardrails, then we are back to square 1. We need a human spending time creating evals and validation scenarios.



imgflip.com

JAKE-CLARK.TUMBLR

5. Individual contribution changes

The shift in individual contribution is subtler than the others, but it compounds.

For junior engineers, agentic coding accelerates ramp-up. Code that once required weeks of pattern learning can be generated and iterated quickly. This is genuinely useful. But if junior engineers are primarily working with generated code without fully understanding what it does, they are building fluency with the interface of AI tools rather than with the domain. When something breaks in production, the debugging requires the deeper understanding they have not developed.

For senior engineers, the dynamic is different. Leverage increases — they can evaluate more, move faster, produce more. But their validation burden increases too. If they are the primary source of “does this actually work?”, and the volume of output has increased significantly, their attention is stretched across more ground with the same capacity.

As [LeadDev notes](#), the shift in engineering roles is from building to evaluating: engineers spending less time writing and more time reviewing, triaging, and validating AI-assisted output.

The reframing for leadership: ownership is not about who wrote the code. It is about who understands it, who will debug it at 2am, and who can explain it to the next engineer who touches it.



The contradictions your team is already living

Here a list of things I have been capturing over the past few weeks speaking to my Data Science teams. They all have the same pattern: we can do X with Claude, but I don't feel more productive.

- We can build faster, but we trust less of what we build.
- We explore more, and we deliver less.
- There is more activity and less completion.
- We generate more code, but we understand less of the system.

- We can technically attempt more things, but prioritisation has become the hardest problem.

This is both exciting but depressing too. The reality is that, as a team lead, you are going to lead the way, with encouragement and resilience.

I want to emphasise how important psychological safety is for your teams. You have to be there saying, “I get it, I don't have all the answers, but I do know we are moving in the right direction”. It's exhausting, I know, so in the next section I propose 6 small things to change to help you run your teams on a day to day basis.

What to do in the next 6 months

There is no framework that resolves these tensions cleanly. The tech is too new to have a golden standard to follow. But there are specific practices that can help reduce the damage.

1. **Cap exploration explicitly.** Define what percentage of sprint capacity can go to spikes, prototypes, and experiments. The number matters less than the existence of a limit. Without it, exploration expands to fill available time and exploitation gets squeezed.
2. **Redefine “done.”** A story is not done when the code is merged. It is done when it is validated, has an owner who understands it, and is observable in production. Tie your definition of done to the validation phase, not the generation phase.

3. **Enforce smaller changesets.** AI-assisted development produces large, dense PRs. Set a soft limit on PR size — not as punishment, but as a design constraint that makes review meaningful. Code that arrives in smaller increments is reviewed better and breaks less.
4. **Strengthen review systems.** Invest in the tooling, time, and standards that make review substantive. Track review throughput as an engineering metric. If review is slowing down, that is a signal, not an inconvenience to route around.
5. **Protect planning discipline.** A prototype is not a commitment. When a stakeholder sees a working POC and asks for a timeline, the answer is not derived from the prototype's velocity. It is derived from what production requires. This needs to be said repeatedly, to different audiences, in different words, until it sticks.



Closing thoughts: the role of leadership in the new triangle

The iron triangle told leaders that their job was to manage scarcity. Time was scarce. Engineering capacity was scarce. Trade-offs were unavoidable, and good leadership meant making those trade-offs deliberately rather than accidentally.

That framing is still partially true. But the scarce *thing* has changed.

What is scarce is the ability to know whether what has been built is correct, maintainable, and worth owning long-term. The new leadership task is not to accelerate everything. Don't try to maximise coverage. You need to maximise impact. You need to decide where acceleration applies, and to be explicit about where it does *not*.

Leadership in this environment is less about optimisation and more about containment and direction. About being willing to say: yes, we could build this in a week, and here is what that means we have decided not to validate, not to own, not to be accountable for.

The hardest part of building software is no longer building fast. It is deciding what deserves to be built carefully.

Now, I want to hear from you

- When have you had to explain to a stakeholder why something takes three months when a demo made it look instant — and what framing actually worked?
- Which part of the reframed triangle has been hardest to make visible in your organisation: trust, control, or planning reliability?
- Are you seeing the exploration-exploitation imbalance in your team? How are you containing it?

Drop your thoughts, questions, or war stories in the comments. I am genuinely curious how others are using Claude Code rules in practice 📌

Further reading

If you are interested in more content, here is an article capturing all my written blogs!



All my articles, one convenient place

JOSE PARREÑO GARCIA · OCTOBER 15, 2024

[Read full story](#)

References

[1] [DORA's software delivery performance metrics](#) — DORA's five software delivery performance metrics and their role as the pre-AI baseline for measuring engineering team performance.

[2] [Generative AI Changes Software Engineering Leaders' Responsibilities](#) — Gartner analysis of how AI is reshaping three core managerial responsibilities: team oversight, talent management, and AI ethics policy.

[3] [Generative AI in software development: A tectonic shift](#) — Insight's analysis of how generative AI structurally reshapes the entire SDLC beyond code generation.

[4] [How has AI impacted engineering leadership in 2025?](#) — Analysis of the 2025 Engineering Leadership Report (617 leaders); 60% report no significant productivity boost from AI.

[5] [Beyond code generation: How AI is changing tech teams' dynamics](#) — Stack Overflow on how AI changes team structure, including the insight that code generation was never the primary bottleneck for shipping software.

[6] [How engineering managers can use GenAI](#) — LeadDev on seven ways engineering managers can apply GenAI to non-coding tasks, with notes on the shift from building to evaluating.

[7] [3 real-world generative AI strategies for executives](#) — Elastic's analysis of the AI adoption gap: nearly 90% of AI projects stuck in pilot mode, fewer than 30% of CEOs actively sponsoring AI initiatives.

[8] [Generative AI for Developers: How to Increase Developer Productivity](#) — Draws on McKinsey controlled research and Stack Overflow 2024 survey; recommends structured enablement and outcome-based measurement over output metrics.

Thanks again for supporting the Senior Data Science Lead newsletter.

Your support makes it possible for me to keep writing openly, while also investing time in the more structured, hands-on work — courses, templates, and tools — that help people apply these ideas in practice. You can find them here.



 50+ Data Science Templates

JOSE PARREÑO GARCIA · APRIL 8, 2025

[Read full story](#)



 Master Data Storytelling

JOSE PARREÑO GARCIA · NOVEMBER 27, 2025

[Read full story](#)

This newsletter exists in its current form because of readers like you. I am genuinely grateful for that.

Thank you.

 LIKE

 COMMENT

 RESTACK

© 2026 Jose Parreño Garcia
548 Market Street PMB 72296, San Francisco, CA 94104
[Unsubscribe](#)

Get the app

 Start writing